

</> ENGINEERING EXCELLENCE

12 Microservices Design Patterns Every Senior Engineer Must Know

An illustrated guide with diagrams and code hints — covering the battle-tested patterns that separate **senior engineers** from the rest.

📌 Most engineers **BUILD** microservices. Few **ARCHITECT** them. Save this. Your teammates will thank you.

12

Core Patterns

Battle-tested architectural
solutions

5+

Companies

Netflix, Uber, Airbnb, Amazon,
Spotify

1

Goal

Architect systems that scale and
survive

Saga Pattern

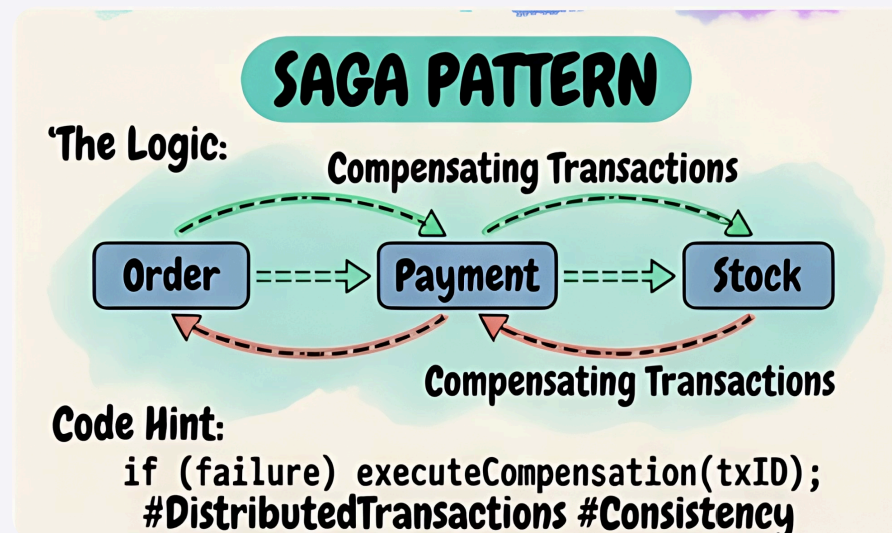
Distributed Transactions Without a DB Lock

The Problem

Multi-service transactions fail halfway → data inconsistency across services with no rollback mechanism.

The Solution

Chain local transactions with **compensating rollbacks**. Each step either succeeds or triggers a reversal.



Two Flavors

- **Choreography** — services react to events (no central brain)
- **Orchestration** — a saga orchestrator commands each step

Strangler Fig Pattern

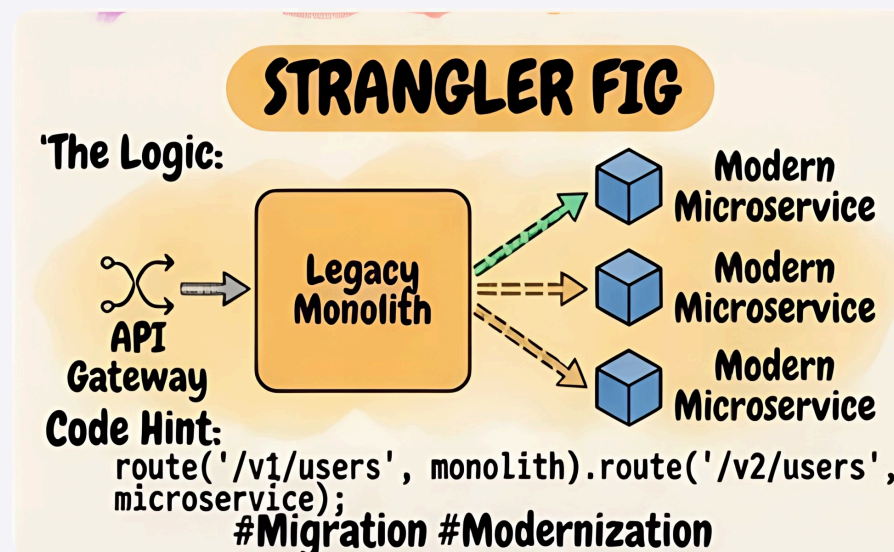
Migrate Legacy Systems Without the Big Bang Rewrite

The Problem

You can't rewrite a monolith overnight. Big bang rewrites fail — they're expensive, risky, and often never finish.

The Solution

Incrementally route traffic to new microservices while the monolith still runs. The old code **dies naturally** as new services take over.



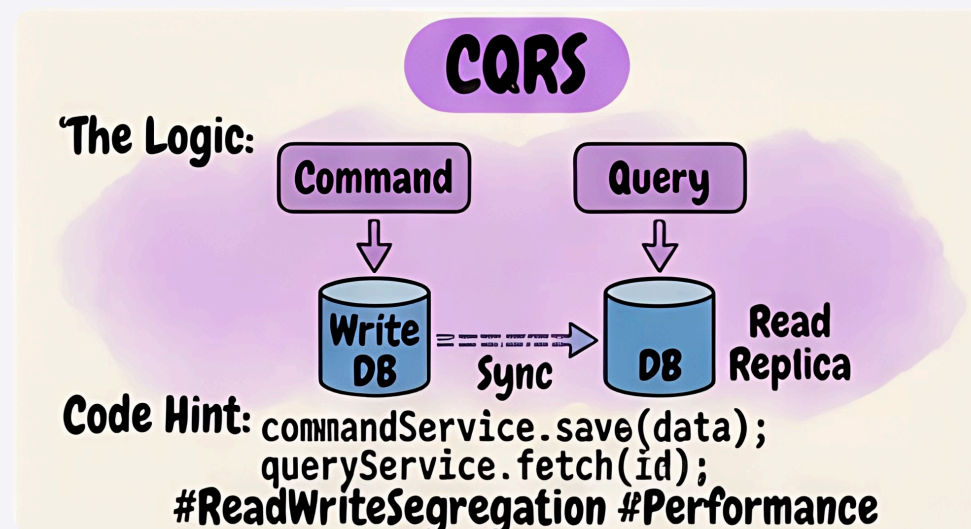
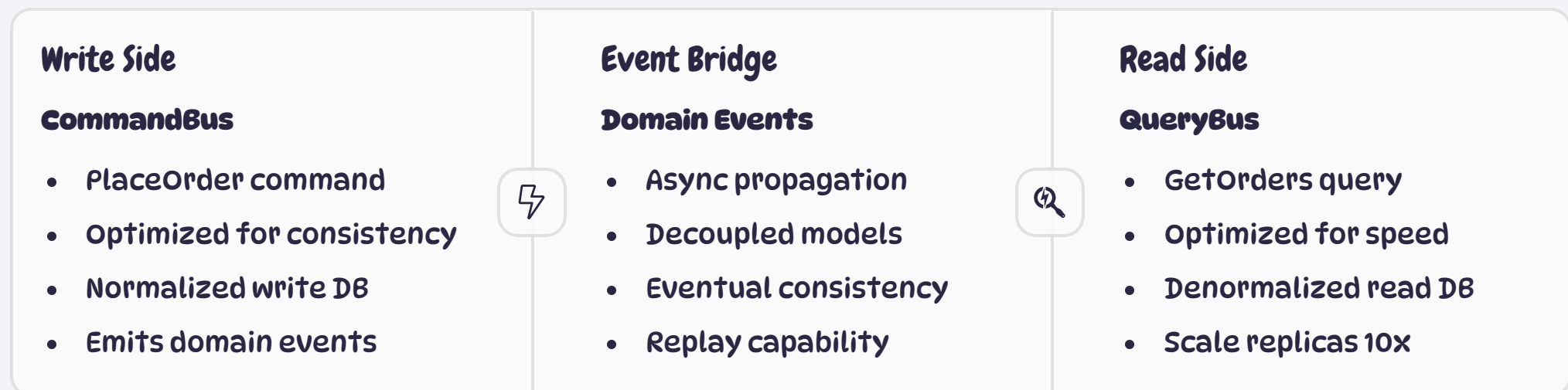
Key mechanism: An **anti-corruption proxy** translates between old and new interfaces.

Used by: Netflix (monolith to AWS migration), Shopify, Booking.com

CQRS – Command Query Responsibility Segregation

Separate Reads from Writes. Scale Each Independently.

Problem: One model handling both reads and writes creates bottlenecks. High read traffic degrades write performance and vice versa.



Event Sourcing

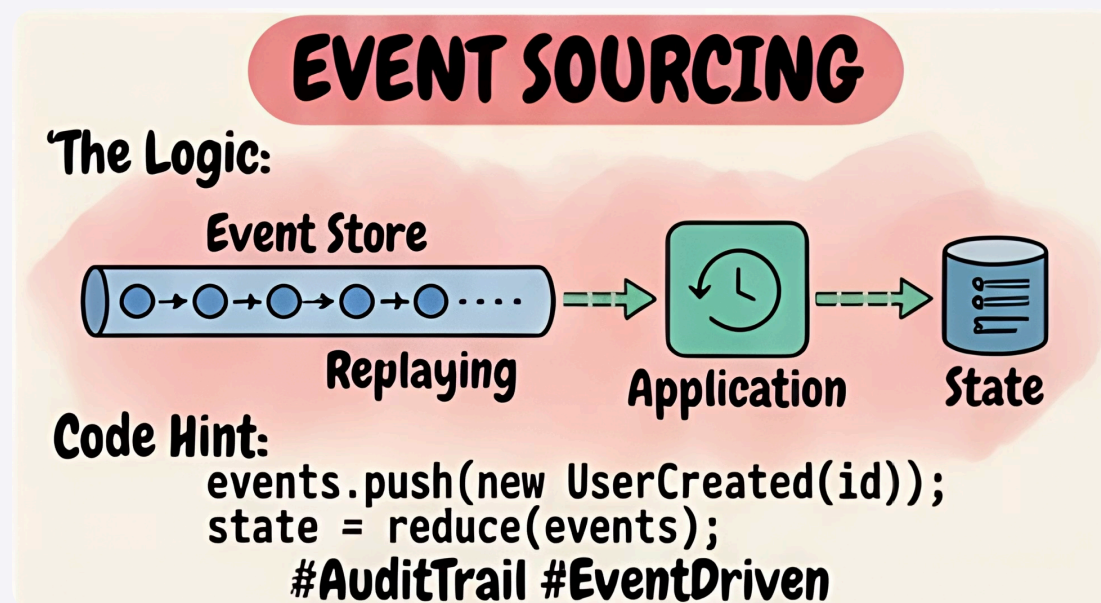
Store Events, Not State. Time-Travel Debugging Included.

The Problem

Traditional DBs store only **current state**. You lose the "why" behind every change — no audit trail, no history, no debugging context.

The Solution

Persist every change as an **immutable event**. Replay events to rebuild state at any point in time.



Superpowers Unlocked

- Complete audit logs
- Time-travel debugging
- Event projections for analytics
- Rebuild any read model from scratch

Outbox Pattern

Guaranteed Event Delivery. No Dual-Write Failures.

Problem: DB write succeeds but message publish fails → **silent data inconsistency** that is nearly impossible to detect or recover from.

✗ Without Outbox (Dangerous)

App → DB Write OK

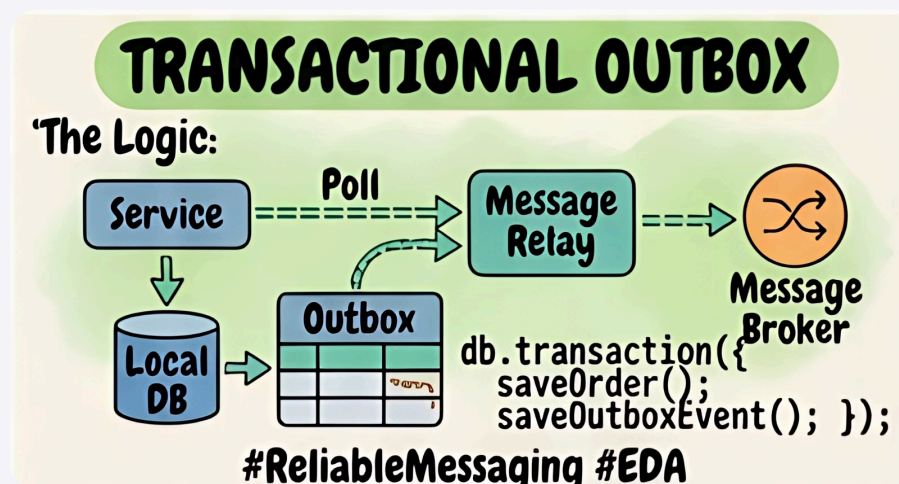
App → Publish Event **FAIL**

Result: **INCONSISTENCY** — DB updated, downstream never notified

✓ With Outbox (Safe)

App → DB Transaction: write business data + write to outbox table in **same transaction**

Relay → reads outbox → publishes to Kafka → marks published



Used by: Debezium (CDC), Spring Transactional Outbox, NestJS

Sidecar Pattern

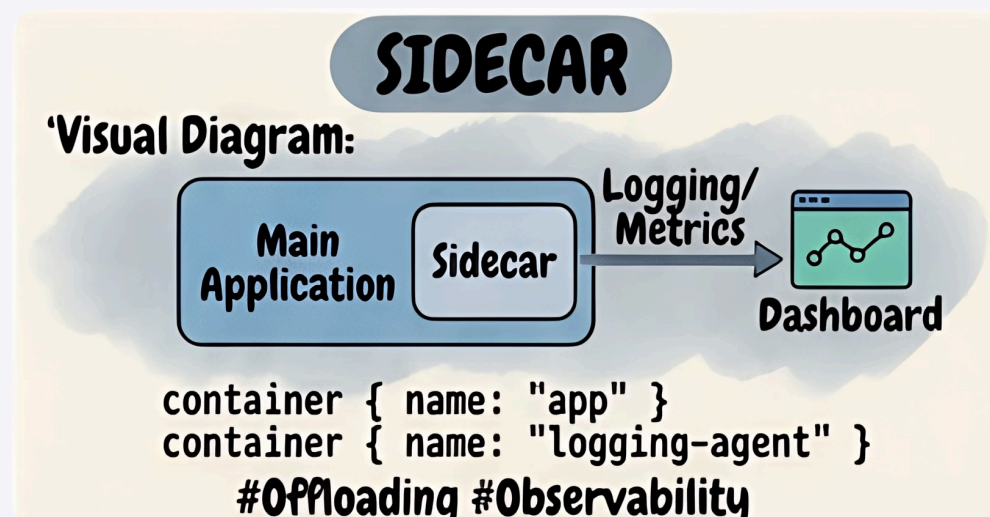
Attach Cross-Cutting Concerns. Zero App Code Changes.

The Problem

Every service re-implements logging, TLS, auth — thousands of lines of **duplicated boilerplate** across every team and every service.

The Solution

Attach a **sidecar container** that handles cross-cutting concerns transparently. The main app doesn't even know it's there.



Used In

Istio

Envoy proxy

Dapr

Runtime sidecar

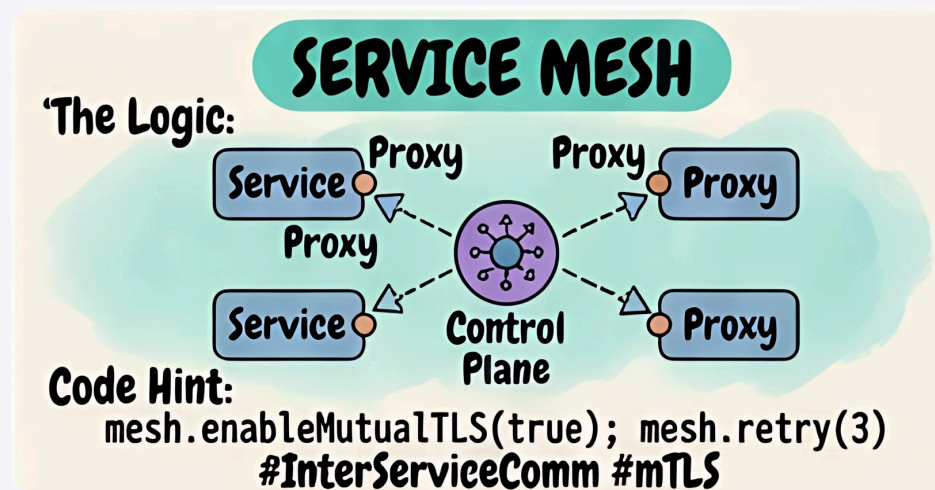
Linkerd

Lightweight proxy

Service Mesh

Observability, Security & Control at Scale

Problem: Implementing tracing, retries, mTLS per service across 100+ services is impossible to maintain consistently.



mTLS Auto-Provisioned

Zero-trust security between every service pair



Distributed Tracing

Full request path visibility across all services



Traffic Splitting A/B

Canary deployments and blue/green with zero code



Circuit Breaking

Automatic failure isolation at the mesh level

Key players: Istio, Linkerd, Consul Connect, AWS App Mesh

API Gateway

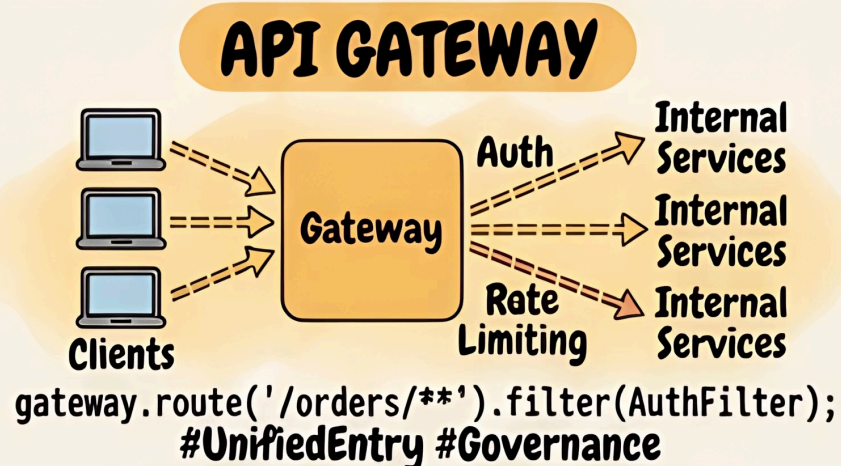
Single Entry Point. Eliminate Client Chaos.

The Problem

Clients calling 20+ services directly → versioning chaos, security gaps, CORS nightmares, and no central control plane.

The Solution

API Gateway = **single front door**. Handles routing, auth, rate limiting, and SSL termination in one place.



Used: AWS API Gateway, Kong, NGINX, Traefik, Apigee

Gateway Capabilities

🔑 Auth – JWT Validation

Centralized authentication before requests reach services

⚡ Rate Limit – 1k req/min

Protect downstream services from traffic spikes

🔄 Routing – /api/*

Intelligent routing to Orders, Users, Products, Payments

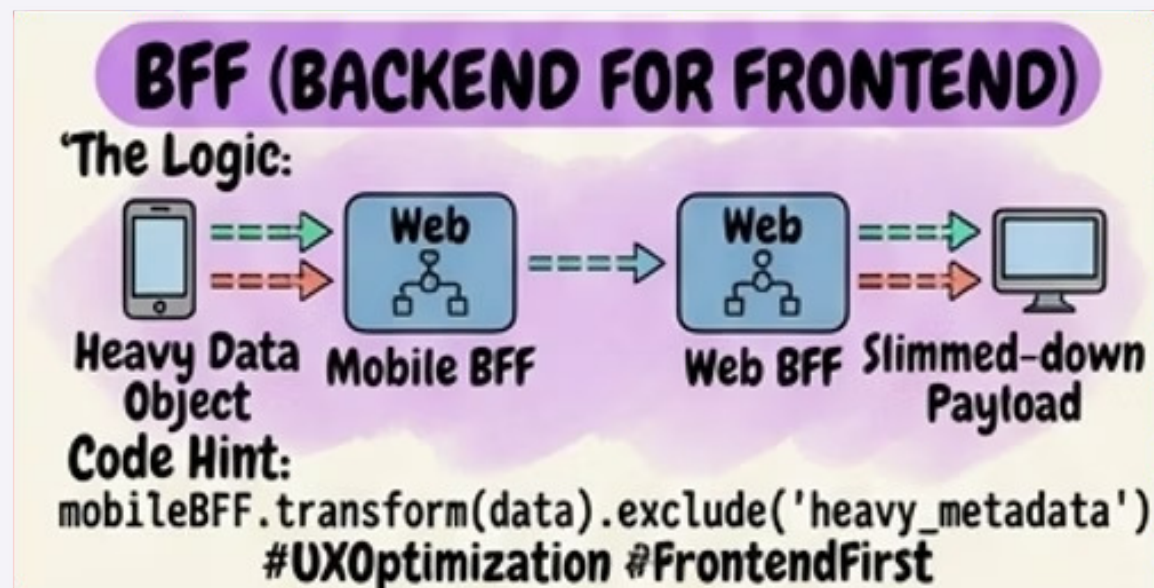
💾 Cache – 60s TTL

Reduce backend load for frequently accessed resources

BFF – Backend for Frontend

One API Can't Serve Everyone. Stop Trying.

Problem: A generic API returns too much for mobile, wrong shape for web, and is useless for IoT. You end up with over-fetching, under-fetching, and frustrated frontend teams.



1

Shared Services

Orders • Users • Products •
Inventory

Single source of truth for all
business data

2

BFF Layer

Mobile BFF — slim, optimized

Web BFF — rich, full data

IoT BFF — minimal, fast

3

Clients

📱 Mobile App

💻 SPA / Web

📡 IoT Device

❏ **Pattern by:** Sam Newman. **Used by:** Netflix, SoundCloud, Spotify — each client gets exactly the data shape it needs, nothing more, nothing less.

PATTERN 10

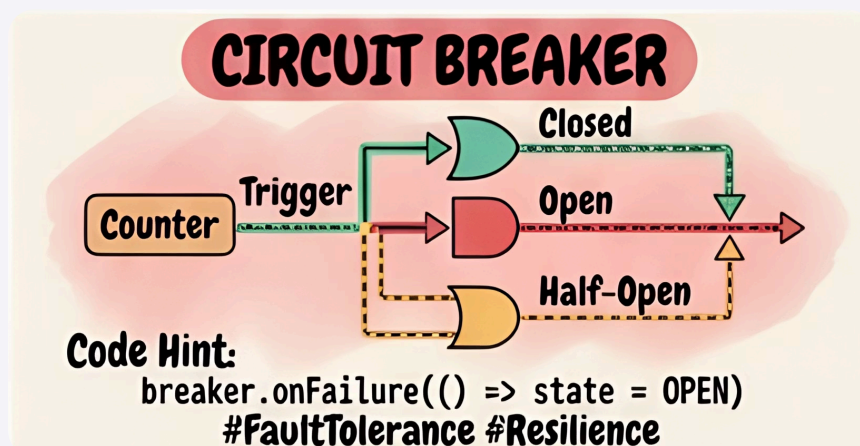
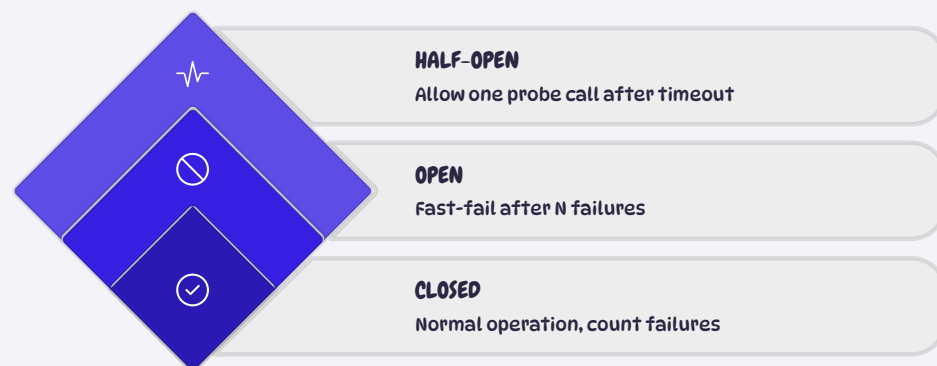
Circuit Breaker

Stop Cascading Failures Before They Kill Your System

The Problem

A slow downstream service causes your service to hang, consuming all threads → **system-wide crash** from a single point of failure.

State Machine



Implementation

```
circuitBreaker
  .call(() => paymentService.charge(order))
  .fallback(() => queueForRetry(order))
  .withThreshold({ failures: 5, window: '10s' })
```

State Behaviors

● CLOSED

Normal operation — all calls pass through to downstream service

● OPEN

Fast-fail mode — no calls made, fallback executed immediately

● HALF-OPEN

Recovery probe — one test call determines if service recovered

Libraries: Resilience4j, Polly (.NET), Opossum (Node.js)

PATTERN 11

Bulkhead Pattern

Isolate Failures. One Leak Shouldn't Sink the Ship.

Problem: One slow service monopolizes your thread pool → all other services starve → total outage from a single dependency.

Without Bulkhead ❌

- ❑ Thread Pool [all 20 threads] ← Payment hangs all 20 → **system down**

With Bulkhead ✅

Payment Pool

4 threads – isolated failure

Inventory Pool

4 threads – stays healthy

Shipping Pool

4 threads – stays healthy

Default Pool

8 threads – stays healthy

Java Implementation

BULKHEAD



Code Hint:

```
executor.setPoolSize(5); // Isolating thread pools  
#Isolation #Stability
```

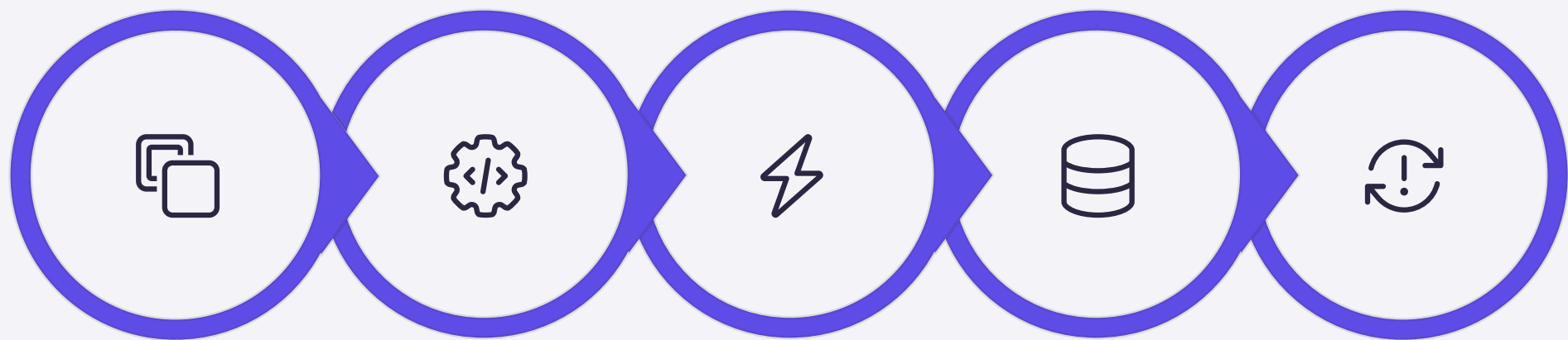
The Metaphor

Named after **ship hull compartments** – watertight sections that prevent flooding from spreading throughout the vessel. One breach doesn't sink the ship.

CQRS + Event Sourcing Combined

The Power Combo for Scalable, Auditable Systems

When combined, these two patterns create something **extraordinary** — a system that is simultaneously scalable, fully auditable, and infinitely flexible.



Command

Aggregate

Events

Event Store

Projector

Financial Systems

Every transaction immutably recorded and auditable

E-Commerce

Order history, inventory projections, customer views

Gaming Leaderboards

Real-time projections from event streams

Healthcare Audits

Compliance-ready immutable patient event logs

❏ **Tradeoff:** Eventual consistency on reads — worth it at scale.

The Pattern Selection Cheat Sheet

Use this table to quickly identify which pattern to reach for when facing a specific architectural challenge.

Problem	Pattern
Distributed transaction across services	Saga
Migrating a legacy monolith	Strangler Fig
Read/write performance bottleneck	CQRS
Need full audit trail & history	Event Sourcing
Event publish failures / dual-write	Outbox
Shared cross-cutting concerns	Sidecar
Service-to-service security at scale	Service Mesh
Too many direct client calls	API Gateway
Different clients need different APIs	BFF
Cascading failures from slow services	Circuit Breaker
Thread pool starvation	Bulkhead
Scalable + fully auditable system	CQRS + Event Sourcing

Save This. Your Future Self Will Thank You. 🚀

You just learned 12 patterns that power the world's most resilient distributed systems.

Netflix Circuit Breaker, Sidecar, Service Mesh	Uber Saga, CQRS, Event Sourcing	Airbnb Saga, BFF, API Gateway	Amazon Saga, Outbox, Bulkhead
Spotify BFF, Service Mesh, Strangler Fig			

The difference between junior and senior engineers is not syntax or frameworks. It is knowing **which pattern to reach for**.

+ Follow for More
Weekly deep-dives on system design, distributed systems, and engineering at scale.



Sameer Paradkar

System Design & Architecture